DoConnect

Q&A Forum Platform

Sprint 1 – Project Report

Great Learning Capstone Project
Wipro Training Program

Submitted by:
Miriyala Chandra Sekhar

GitHub: `https://github.com/mchandu454/DoConnect_Capstone_Project`

March 2026

Chapter 1

Project Overview

DoConnect is a full-stack Question and Answer (Q&A) platform implemented as part of the Great Learning Capstone project (Sprint 1). The application enables users to post questions, submit answers, and browse existing discussions, while an administrator oversees and moderates activity.

The solution is built with a clear separation between:

- an ASP.NET Core Web API back-end that exposes RESTful endpoints and handles data access, business logic and security, and
- an Angular front-end that provides a responsive single-page user interface.

The primary Sprint 1 goals were:

- design and implement the core database schema for users, questions, answers and images,
- build secure APIs for authentication and Q&A operations,
- create the Angular front-end with routing and service integration,
- configure JWT-based authentication end-to-end,
- and introduce basic back-end unit tests using xUnit and Moq.

1.1 Technology Stack

Core Technologies

Layer	**Technology**	**Version**

Backend Framework	ASP.NET Core Web API (.NET)	.NET 8.0
Frontend Framework	Angular	21.x
Database	Microsoft SQL Server	Local / Express instance
ORM	Entity Framework Core (SQL Server provider)	8.0.5
Authentication	JWT Bearer Authentication	8.0.0 package version
Real-time Client	SignalR JavaScript Client (@microsoft/signalr)	10.0.0
API Documentation	Swagger via Swashbuckle.AspNetCore	6.6.2
Back-end Unit Testing	xUnit, Moq, EF Core InMemory	xUnit 2.9.3, Moq 4.20.72, InMemory 10.0.5
Angular Tooling / Testing	Angular CLI, TypeScript, Vitest, jsdom	CLI 21.2.x, TS 5.9.x

1.2 Solution Structure

Folder Layout (High Level)

Folder / Project	Description
`DoConnect.API/`	ASP.NET Core Web API project (back-end).
`DoConnect.API/Controllers/`	Controllers exposing HTTP endpoints for authentication, questions and answers.
`DoConnect.API/Models/`	Entity classes mapped to database tables (User, Question, Answer, Image).
`DoConnect.API/Data/`	Entity Framework Core ApplicationDbContext, relationship configuration and initial seeding (admin user).
`DoConnect.API/DTOs/`	Data Transfer Objects used to exchange data between API and front-end.
`DoConnect.API/Repositories/`	Repository interfaces and implementations for data access.
`DoConnect.API/Services/`	Application service layer encapsulating business logic.
`Doconnect-fronted/`	Angular workspace containing the front-end application.
`Doconnect-fronted/src/app/components/`	Angular components for pages and reusable UI.
`Doconnect-fronted/src/app/services/`	Angular services that call the back-end APIs.
`Doconnect-fronted/src/app/guards/`	Route guards for protecting authenticated routes.
`Doconnect-fronted/src/app/interceptors/`	HTTP interceptors for attaching JWT tokens and handling responses.
`DoConnect.API.Tests/`	xUnit test project for back-end unit tests.

Chapter 2

System Diagrams

2.1 Use Case Diagram

2.2 ER Diagram

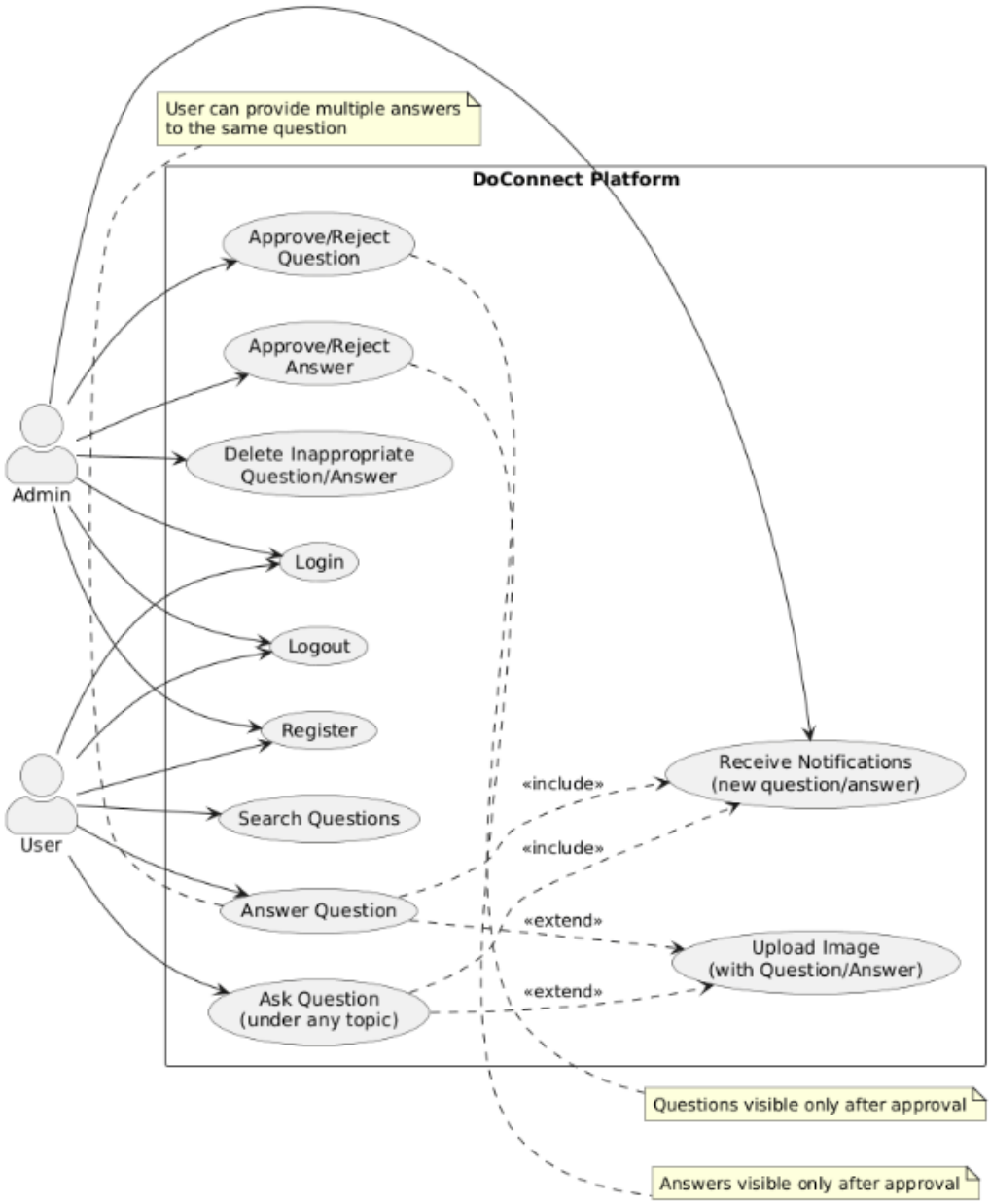

Figure 2.1: DoConnect Use Case Diagram

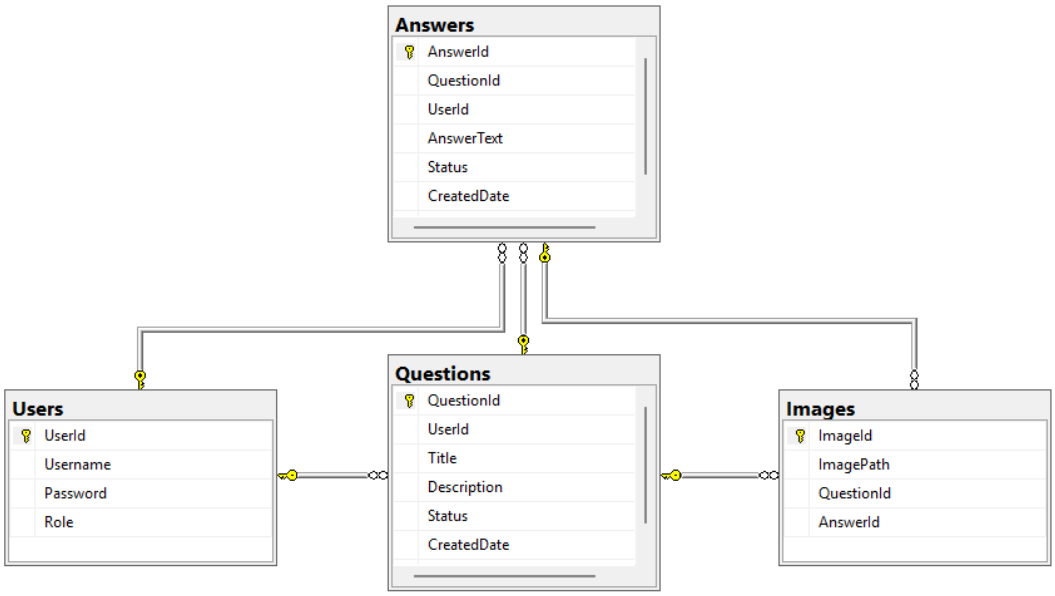

Figure 2.2: DoConnect ER Diagram

Chapter 3

Database Design

The back-end uses SQL Server together with Entity Framework Core to manage persistence. The initial schema in this sprint focuses on storing users, their questions, answers, and any image paths associated with those questions or answers.

3.1 Tables and Relationships

Main Tables

Table	Primary Key	Description
Users	UserId	Stores application users, including the seeded administrator account.
Questions	QuestionId	Stores questions posted by users, including title, description, status and timestamps.
Answers	AnswerId	Stores answers that users write for questions, with status and timestamps.
Images	ImageId	Stores image path information, optionally linked to a question or an answer.

Configured Relationships

The relationships are configured in `ApplicationDbContext` using the Fluent API:

- Users (1) → Questions (M)
- Users (1) → Answers (M)
- Questions (1) → Answers (M)

- Questions (1) → Images (M)
- Answers (1) → Images (M)

Delete behaviour is carefully controlled (combination of cascade and restrict) to avoid multiple cascade paths and to preserve required data.

Chapter 4

Features Implemented

This section summarises the Sprint 1 functionality delivered for DoConnect, grouped by high-level feature area.

4.1 Authentication and Authorization

- User registration with basic validation of credentials.
- User login that returns a JWT token on successful authentication.
- JWT-based protection of secured API endpoints using the `JwtBearer` authentication scheme.
- A seeded admin account in the database with elevated privileges.
- Role-based behaviour on the back-end (for example, admin moderation actions versus regular user actions).

4.2 Question Management

- Authenticated users can create questions with title, description and an optional image path.
- Questions have a `Status` field (`Pending`, `Approved`, `Rejected`, `Deleted`) that is used for moderation and history views.
- Repositories provide paging and filtering capabilities:
  - get all approved questions with pagination,
  - search approved questions by keyword,
  - list pending questions for moderation,

  - list questions by user and history views.

- Users can retrieve detailed information for a question including creator details, answers and images.
- Ownership checks ensure that only the user who posted a question can edit or mark it as deleted via the user-facing endpoints.

4.3 Answer Management

- Users can submit answers to existing questions.
- Each answer is tied to a specific question and the user who posted it.
- Answers support status values so they can be moderated similarly to questions.
- Image metadata can be associated with answers using the shared `Images` table.

4.4 Admin Capabilities

- The admin account is seeded via EF Core model configuration.
- Admin-specific workflows allow:
  - reviewing pending questions,
  - reviewing history of moderated questions,
  - and marking questions/answers as deleted or changing statuses.

4.5 Angular Front-end

- Angular 21 application created using the latest Angular CLI.
- Application routing to navigate between login, registration, question list and question detail screens.
- Angular services (for example, `question.service.ts`) encapsulate calls to the ASP.NET Core API.
- Forms are used for creating and updating questions and answers, with basic client-side validation.
- The Angular project is configured with modern tooling (TypeScript, Vitest, jsdom and Prettier).

Chapter 5

CRUD Operations for Each Model

This chapter explicitly summarises the Create, Read, Update and Delete (CRUD) operations implemented for each major model in the DoConnect system.

5.1 User Model (Users Table)

Create

- **Register User** – via `POST /api/auth/register`. Creates a new user record with username, password and role (default user).
- **Seed Admin** – configured in `ApplicationDbContext` using `HasData` to insert an initial admin record.

Read

- **Get Current User** – `GET /api/auth/me` returns the logged-in user's details based on the JWT token.
- **Get Users (Admin)** – admin-specific endpoints (or repository methods) allow listing all users for moderation or reporting.

Update

- **Update User Status / Role (Admin)** – admin can change user properties such as activation status or role (where implemented in the admin controller/service).

Delete

- **Soft / Logical Removal** – instead of physically deleting users, the system can deactivate accounts or prevent login while preserving related questions and answers.

5.2 Question Model (Questions Table)

Create

- **Create Question** – `POST /api/questions`. Uses `CreateQuestionDto` and `QuestionService.Cr` to create a new question linked to the current user and optional image path.

Read

- **Get Question By Id** – `GET /api/questions/{id}`. Returns detailed information about a single question, including user, answers and images.
- **Get All Approved Questions** – `GET /api/questions`. Uses pagination in `QuestionRepository`
- **Search Approved Questions** – `GET /api/questions/search?query=...`. Uses `SearchAsync` and `CountSearchAsync`.
- **Get Pending Questions** – `GET /api/questions/pending`. For admin moderation, implemented via `GetPendingAsync`.
- **Get History** – `GET /api/questions/history`. Shows approved, rejected and deleted questions.
- **Get Questions By User** – e.g., `GET /api/questions/user/{userId}`. Uses `GetByUserIdAsync`.

Update

- **Update Own Question** – `PUT /api/questions/{id}`. Uses `QuestionService.UpdateOwnAsyn` which:
  - checks ownership (only the creator can modify),
  - updates title, description and optional image path through `QuestionRepository.UpdateAsy`
- **Admin Status Updates** – admin operations can change the `Status` field (e.g., approve or reject) using repository/service methods.

Delete

- **Delete Own Question** – `DELETE /api/questions/{id}`. Implemented via `QuestionService.D` which:
  - verifies that the logged-in user owns the question,

  - sets status to `"Deleted"` and calls `QuestionRepository.DeleteAsync`.

- **Admin Delete** – admin can also mark inappropriate questions as deleted through moderation endpoints.

5.3 Answer Model (Answers Table)

Create

- **Create Answer** – `POST /api/answers`. Creates a new answer for a specific question, linked to the currently logged-in user.

Read

- **Get Answers For Question** – `GET /api/answers/question/{questionId}`. Returns all answers belonging to a given question, often including user information.

Update

- **Update Own Answer** – `PUT /api/answers/{id}`. Only the user who posted the answer is allowed to edit its text or associated image (enforced in the service layer).

- **Admin Status Updates** – admin can approve, reject or mark answers as deleted depending on moderation requirements.

Delete

- **Delete Own Answer** – `DELETE /api/answers/{id}`. Sets the answer status to a deleted state (soft delete) rather than physically removing it, to preserve history.

- **Admin Delete** – admin can remove inappropriate answers by changing the status through dedicated endpoints.

5.4 Image Model (Images Table)

Create

- **Attach Image to Question** – When a question is created with an `imagePath`, `QuestionService.CreateAsync` creates an `Image` entry linked to the question.

- **Attach Image to Answer** – Similar logic applies when answers support images; an `Image` record is associated with the answer.

Read

- **Load Images with Questions** – `QuestionRepository.GetByIdAsync` and other methods include `Images` using `Include`, so the client can display question images.
- **Load Images with Answers** – Navigation from `Answer` to `Images` allows loading answer images.

Update

- **Update Image Reference** – When a question or answer is edited with a new image, the corresponding `ImagePath` and relationships can be updated in the database.

Delete

- **Cascade Delete** – If a question is removed, its related images are deleted via cascade; when an answer is deleted, answer-linked images are either restricted or deleted based on configured behaviour in `ApplicationDbContext`.

Chapter 6

API Endpoints

All endpoints are implemented in the controllers under `DoConnect.API/Controllers`. The base URL for the back-end is typically `https://localhost:{port}`.

6.1 Authentication Endpoints (`/api/auth`)

Method	**Endpoint**	**Description**
POST	`/api/auth/register`	Register a new user using `RegisterDto`.
POST	`/api/auth/login`	Login with username and password, returns JWT token and user info.

6.2 Question Endpoints (`/api/questions`)

Method	**Endpoint**	**Description**
GET	`/api/questions`	Get all **approved** questions with pagination (`page`, `pageSize`).
GET	`/api/questions/{id}`	Get full details for a single question (user, answers, images).

GET	`/api/questions/search?q={query}`	Search approved questions by keyword with pagination.
GET	`/api/questions/my-questions`	(Auth) Get all questions created by the logged-in user.
POST	`/api/questions`	(Auth) Create a new question (title, description, optional image).
PUT	`/api/questions/{id}`	(Auth) Update the logged-in user's own question (with optional new image).
DELETE	`/api/questions/{id}`	(Auth) Soft-delete the logged-in user's own question (status set to `Deleted`).

6.3 Answer Endpoints (`/api/answers`)

Method	**Endpoint**	**Description**
GET	`/api/answers/question/{questionId}`	Get all answers for a specific question.
GET	`/api/answers/my-answers`	(Auth) Get all answers written by the logged-in user.
POST	`/api/answers`	(Auth) Create a new answer for a question (text + optional image).

PUT	`/api/answers/{id}`	(Auth) Update the logged-in user's own answer (with optional new image).
DELETE	`/api/answers/{id}`	(Auth) Soft-delete the logged-in user's own answer.

6.4 Admin Endpoints (`/api/admin`)

All admin endpoints require the user to have role `Admin`.

Review Lists

Method	Endpoint	Description
GET	`/api/admin/pending-questions`	Get all questions with status `Pending` for review.
GET	`/api/admin/pending-answers`	Get all answers with status `Pending` for review.
GET	`/api/admin/history-questions`	Get history of moderated questions (Approved / Rejected / Deleted).
GET	`/api/admin/history-answers`	Get history of moderated answers.

Moderation Actions

Method	Endpoint	Description
PUT	`/api/admin/approve-question/{id}`	Set question status to `Approved`.

PUT	`/api/admin/reject-question/{id}`	Set question status to `Rejected`.
DELETE	`/api/admin/delete-question/{id}`	Mark question as deleted (admin delete).
PUT	`/api/admin/approve-answer/{id}`	Set answer status to `Approved`.
PUT	`/api/admin/reject-answer/{id}`	Set answer status to `Rejected`.
DELETE	`/api/admin/delete-answer/{id}`	Mark answer as deleted (admin delete).

Chapter 7

Security Implementation

7.1 JWT Authentication

The back-end uses JSON Web Tokens (JWT) to secure protected API endpoints.

- On successful login, the server issues a JWT that encodes key claims such as user identifier and role.
- The Angular client includes this token in the `Authorization` header using the `Bearer` scheme for subsequent requests.
- Token validation is configured in `Program.cs` using `JwtBearerDefaults.AuthenticationScheme`, along with issuer, audience and signing key values from configuration.

7.2 Middleware Pipeline

The ASP.NET Core middleware setup includes:

Middleware	**Purpose**
Static Files	Serves static assets and uploaded images from the `wwwroot` folder (e.g., `/uploads`).
Swagger / Swagger UI	Exposes documentation and interactive testing surface while running in development.
CORS Policy ("AllowAngular")	Allows the Angular front-end (typically running on `http://localhost:4200`) to call the API with appropriate headers and credentials.
Authentication	Validates JWT tokens attached to incoming requests.

Authorization	Enforces access rules for admin and regular users.
Endpoint Routing (MapControllers)	Dispatches incoming HTTP requests to the corresponding controller actions.

Chapter 8

Sprint 1 Requirements Completion

Requirement	**Marks**	**Status**
Database schema, tables and relationships with EF Core	3	Completed
Admin login and basic admin operations on content	3	Completed
User registration and login with JWT authentication	3	Completed
Angular front-end scaffold, routing and views	–	Completed
Layered code structure (Controllers, Services, Repositories)	8	Completed
Core CRUD flows for questions and answers (Admin/User)	8	Completed
Additional enhancements (status-based history, image support)	8	Completed
Responsiveness and basic client-side validation	–	Completed
Session handling with JWT stored on client and validated by API	5	Completed
Integration between Angular and back-end APIs	5	Completed

Back-end unit tests (xUnit, Moq, EF Core InMemory)	7	Implemented for selected services/repositories
Github repository	3	Completed

Chapter 9

GitHub Repository

Repository Link: `https://github.com/mchandu454/DoConnect_Capstone_Project`

- `DoConnect.API/` – ASP.NET Core Web API back-end
- `Doconnect-fronted/` – Angular front-end
- `DoConnect.API.Tests/` – Unit tests (xUnit)